

Hanwella WMS QA Audit

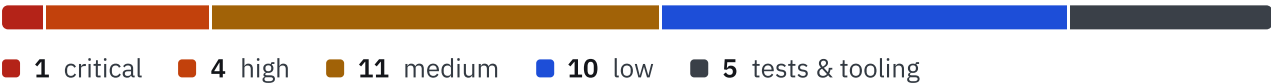
Date 11 Sep 2026 Build commit 72eb382 + uncommitted Security module

Stack Laravel 12 · PHP 8.5 · MySQL · React 18 Size 9.4k lines PHP · 12.6k lines JS

NOT READY FOR GO-LIVE

Strong foundations, four blockers

The core design is careful: every stock change is row-locked, prices are resolved on the server, and one permission check guards all 74 API routes. But one bug stops the counter selling any stock that sits in a bin — which today is **all of it**. Editing a part quietly erases data. Database errors leak to the browser, and there is no supported way to create the first production admin.



Checks run

Destructive probes ran against the isolated `hanwella_wms_test` database. The one request sent to the dev API was a sale the server refused and rolled back.

CHECK	RESULT	NOTES
Backend feature tests (Pest)	106 / 106 PASS	431 assertions across 9 files. Needs <code>--testsuite=Feature</code> — see T1.
Frontend tests (Vitest)	47 / 47 PASS	9 files; large parts of the app untested — see T5.
ESLint · production build	CLEAN	Vite build completes in under 2 s.
<code>composer audit</code>	0 ADVISORIES	laravel/framework v12.69.2.
<code>npm audit</code>	7 ADVISORIES	Only react-router-dom ships to users (moderate). The critical/high items are dev tooling — see M11.
Live API probes	14 DEFECTS	~40 crafted requests: bounds, bad dates, state transitions, admin guards, throttling, deactivation.
Concurrency (real processes)	HELD	8 processes dispatched one order at the same instant: exactly 1 succeeded.

CHECK	RESULT	NOTES
Browser walkthrough	NO ERRORS	17 screens of the Security module in Chrome, desktop and 400 px, with no console errors.

Recommended fix order

Sequenced by what blocks trading first, then what corrupts data, then what's cheap.

1	Make binned stock sellable before anyone rings up a sale. C1
2	Stop the part-edit data loss. Lost fitment can be restored from <code>audit_logs.old_values</code> on <code>part.update</code> rows. H1
3	Fix the exception mapping in <code>bootstrap/app.php</code> : one edit closes the DB-error leak and the 500s. H2 · M1 · M3
4	Add <code>wms:create-admin</code> and a last-admin guard. Go-live depends on it. H3
5	Switch to sequential order numbers while there is still no real sales data to migrate. H4
6	Quick wins: company details on bills, clear the cart on sign-out, <code>storage:link</code> , Colombo timezone. M5 · M6 · M7 · M9
7	Product decisions: discount limits, real settings, a payment ledger. M2 · M4 · M8
8	Add CI with the full test run so none of this regresses. T1 – T5

Findings

Reproduced live means confirmed with a real request; everything else is from code review.

CRITICAL Stops the business from trading

c1 Stock held in bins can't be sold or issued

Stock · Sales

Reproduced live

The inventory screen shows a part in stock, but the counter sale and stock-out both refuse it as having **0 on hand**. In the current dev data, **all 9,955 units are stored in bin locations**: 228 parts show stock, and **0 can actually be sold**.

```
GET /api/v1/inventory?status=in
200 BRK-SUZ-3361 · quantity 58

POST /api/v1/orders {"items":[{"part_id":1,"quantity":1}], "payment_status":"PAID"}
409 "Insufficient stock for BRK-SUZ-3361. 0 unit(s) on hand, 1 requested."

POST /api/v1/stock/out {"part_id":1,"quantity":1}
409 "Insufficient stock for BRK-SUZ-3361. 0 unit(s) on hand, 1 requested."
```

CAUSE `StockService::lockRow()` — with no location given — only ever touches the row where `location_id IS NULL` in the first active warehouse. `OrderService` never passes a location, and the Stock Out form sends only `warehouse_id`. Stock placed in a bin (seed data, a part created with a location, a stock-in to a bin) is unreachable from the UI.

FIX With no location, draw across the part's rows in that warehouse: lock them in id order, take from the unbinned row first, then the fullest bin. Or have the sale/issue screens pick a bin. Report the true on-hand figure in the error, and add a test that sells binned stock — every current stock test uses unbinned stock, which is why this passed.

HIGH Loses data, leaks internals, or blocks go-live

H1 Saving a part from the edit form erases its vehicle fitment

Catalogue

Reproduced live

Open any part, change nothing, press Save: the server answers **200 OK** and the part loses its make and model. The vehicle and bin that staff type when adding a part are also silently discarded.

```
# part 240 "Mud Flap Set - Suzuki Baleno"
before  vehicle_model_id: 27  make: Suzuki  model: Baleno
PUT /api/v1/parts/240 (exact fields PartFormPage sends – nothing changed)
200 "Spare part updated successfully."
after   vehicle_model_id: null make: null    model: null
```

CAUSE `PartController::update` sets `'vehicle_model_id' => $data['vehicle_model_id'] ?? null`. The form (`PartFormPage.jsx`) sends free-text `vehicle_make`, `vehicle_model` and `bin`. `PartRequest` doesn't accept those, so `validated()` drops them.

FIX Use make/model pickers that send `vehicle_model_id`, and a bin picker that sends `location_id`. In update, only change keys actually present in the request. Restore affected parts from `audit_logs.old_values`.

H2 Database errors are sent to the browser, even with `APP_DEBUG=false`

Security · Error handling

Reproduced live

Any database error reaches the client verbatim, including the SQL, table and column names, and the DB host, port and schema. The deployment checklist's "`APP_DEBUG=false` returns the generic message" does not hold for these.

```
GET /api/v1/orders?page=-5
422 "SQLSTATE[42000]: Syntax error or access violation: 1064 ...
    (Connection: mysql, Host: 127.0.0.1, Port: 3306, Database: ..., SQL: select ...)"
POST /api/v1/stock/in {"part_id":2,"quantity":5000000000}
422 "SQLSTATE[22003]: Numeric value out of range: 1264 ... column 'quantity'"
```

CAUSE `bootstrap/app.php` returns every `RuntimeException`'s message as a 422 with no debug check. Laravel's `QueryException` extends `PDOException`, which extends `RuntimeException`.

FIX Map `QueryException` to a generic 500 (logged in full) ahead of that branch. Better still, use a dedicated user-facing exception for intentional 422s instead of catching all `RuntimeException`s.

H3 No way to create the first production admin — and the only admin can lock everyone out

Users · Deployment

Reproduced live

`UserSeeder` says production admins are made with `php artisan wms:create-admin`, but that command doesn't exist. The deployment guide falls back to "tinker, or a dedicated artisan command, once one exists". Meanwhile the sole admin can demote themselves, after which nobody can manage users.

```
PUT /api/v1/users/{own id} {"role":"VIEWER"} # the only ADMIN
200 "Account updated successfully."
GET /api/v1/users
403 "You do not have permission to perform this action."
```

FIX Implement `wms:create-admin`. Refuse role changes or deactivation that would leave zero active admins, including your own account.

H4 Order numbers run out at 9,999 a year, and aren't sequential

Sales

Code review

`nextOrderNo()` picks `S0-2026-` plus a **random** number from 1 to 9,999, and retries 5 times on a collision. With N numbers already used, a sale fails with probability $(N/9,999)^5$. At 30 sales a day (~11,000 a year) the counter starts refusing sales in the fourth quarter. Random numbers also break the gap-free bill sequence that auditors and tax invoices expect.

ORDERS THIS YEAR	CHANCE A SALE FAILS
5,000	3%
8,000	33%
9,000	59%
9,999	100%

FIX A row-locked per-year sequence, `S0-2026-000123`, and widen `order_no`. Change it now, while there's no real sales data to migrate.

M1 Invalid order actions crash with HTTP 500

Sales

Reproduced live

```
POST /api/v1/orders/{id}/cancel    # already cancelled
500  "This order is already cancelled."
PATCH /api/v1/orders/{id}/payment # cancelled order
500  "A cancelled order cannot have its payment changed."
```

CAUSE `OrderService` throws `InvalidArgumentException`, a `LogicException` the handler doesn't map, so it becomes a 500. In production the user sees only "An unexpected error occurred."

FIX Throw a domain exception that renders as 409, the way the new dispatch code does.

M2 A cashier can discount any order to zero

Sales · Controls

Reproduced live

The discount is clamped to the subtotal, so an oversized discount produces a free order marked **PAID**. That order is immediately eligible to leave the yard. There's no cap, no approval step and no separate permission.

```
POST /api/v1/orders {"discount":10000000000000, ...} # as warehouse staff
201  "Order S0-2026-1007 completed successfully." total Rs 0 · PAID
```

FIX A maximum discount percentage; anything above it needs a manager permission; flag large discounts on the dashboard and in the audit log.

M3 List and stock endpoints accept out-of-range input

Validation

Reproduced live

A negative `per_page` or a quantity above `INT UNSIGNED` reaches MySQL and fails (and leaks — H2). Invalid dates are accepted silently: `from=gabrage` on `/orders`, `/movements`, `/audit-logs` and `/reports/*` returns 200 with a meaningless filter.

FIX One shared list request: `per_page` 1–100, and `from / to` as `date_format:Y-m-d` (the Security endpoints already do this). Give quantity fields a sane maximum.

M4 Most of the Settings page has no effect

Settings

Code review

Nothing outside `SettingsController` reads the `settings` table. QR prefix and padding (`QrService` reads `.env`), currency, default warehouse, default minimum stock and all three email-notification toggles save successfully and change nothing. No mail or notification code exists.

FIX Load settings into config at boot through one settings service, or hide the controls until they're wired up.

M5 Bills printed by non-admins show placeholder company details

Sales · Printing

Code review

`BillPage` reads the company block from `GET /settings`, which requires `manage_settings`. For cashiers it returns 403, and the bill falls back to a hard-coded name with no address, contact or registration number — on a customer-facing document.

FIX Serve the company block from an endpoint any signed-in user can read, or include it in the order response.

M6 A half-built sale survives sign-out on a shared till

Sales · Privacy

Code review

The basket, including the customer's name and phone, is kept in the browser as `wms.cart`. Sign-out clears the data cache but not the basket, so the next cashier inherits the previous customer's details and lines.

FIX Clear the cart on sign-out and on session expiry, or key it to the signed-in user.

M7 Uploaded part photos 404 on a fresh install

Setup

Confirmed

Photos are stored on the `public` disk and linked through `/storage`, but `backend/public/storage` doesn't exist. The README install steps omit `php artisan storage:link`, and the deployment guide calls it "only needed if" — it's always needed.

M8 Payments have no history

Finance · Design

Code review

Each order holds a single, overwritable `paid_amount`. There is no record of individual instalments — who took them, when, and by which mode. "Mark pending" on a paid order resets the amount to 0. The only trail is the audit log.

FIX A payments ledger (order, amount, mode, received by, received at), with `paid_amount` derived from it.

M9 "Today" and daily reports run on UTC, not Sri Lanka time

Reporting

Code review

`config/app.php` hard-codes `'timezone' => 'UTC'`. Sales between 00:00 and 05:30 in Colombo land on the previous day in the dashboard, the sales-by-day report and "dispatched today", and every date filter is shifted 5½ hours.

FIX Make it `env('APP_TIMEZONE', 'Asia/Colombo')`, or convert at query time.

M10 CSV exports can carry spreadsheet formulas

Reports · Security

Code review

Reports are exported with plain `fputcsv`. The low-stock report writes part names and numbers, which come from forms and bulk imports. A value starting with `=`, `+`, `-` or `@` runs as a formula when a manager opens the file in Excel.

FIX Prefix any cell that starts with one of those characters with a single quote.

M11 Front-end router has a published advisory

Dependencies

npm audit

`react-router-dom` 6.28 (moderate: an open redirect via a backslash in `<Link> / navigate`). The critical and high advisories — vitest ≤ 4.1.10, vite ≤ 6.4.2, esbuild — affect the local dev server only and don't ship to users.

FIX Upgrade the router to a patched release. Move to vitest 5 for dev.

- L1 One mistyped password can lock the whole warehouse out**
Login is throttled to 6 attempts a minute *per IP*. Behind one shared connection, a single user's wrong passwords block every account for 60 s (reproduced: six 422s, then a 429 for a different account). Throttle per email + IP instead.
-
- L2 Deactivated users stay "signed in" to an empty app**
Deactivation takes effect immediately — every protected call returns 403 (reproduced). But `/auth/me` still returns 200, so the UI keeps the session open instead of signing out with a message.
-
- L3 No security headers; PHP version exposed**
Responses carry `X-Powered-By: PHP/8.5.10` and no CSP, `frame-ancestors`, HSTS, `nosniff` or `Referrer-Policy`. Add them at the web server or in middleware.
-
- L4 Production CORS still admits a dev origin**
`config/cors.php` always includes `http://127.0.0.1:5173` with credentials, whatever `FRONTEND_URL` says.
-
- L5 404 vs 403 reveals which record IDs exist**
Route-model binding runs before the permission check, so an unauthorised user gets 404 for missing IDs and 403 for real ones.
-
- L6 Stock-in accepts a bin from a different warehouse**
`StockMoveRequest` checks that both IDs exist, not that the location belongs to the warehouse, so it can create a mismatched inventory row.
-
- L7 "Default warehouse" is whichever row MySQL returns first**
`defaultWarehouseId()` has no `orderBy`, so it becomes unpredictable once there are two active warehouses (and the settings default is ignored — M4).
-
- L8 Two screens will slow as the catalogue grows**
The inventory list loads every matching part on each request to count its tabs. QR labels run one query per label (up to 200). Fine at 240 parts; not at 20,000.
-
- L9 A deleted part's number can't be reused**
Soft-deleted parts keep their unique `part_number` and SKU, and the validation rule doesn't exclude deleted rows.

L10 Looking up a QR code counts as a scan

`GET /qr/{code}` increments `scan_count` , so page refreshes and prefetching inflate scan statistics.

TESTS & TOOLING Why defects like C1 get through

T1 `php artisan test` stops before running anything

`phpunit.xml` lists a `tests/Unit` suite that doesn't exist, so the default command (and `composer test`) exits with "Test directory not found". It works only with `--testsuite=Feature` .

T2 Test database credentials are hard-coded

`phpunit.xml` pins `root / root` . Also, a cached config makes the suite's `RefreshDatabase` wipe the *dev* database — always run `config:clear` first.

T3 No continuous integration

There is no `.github/` workflow. Tests, lint and audits run only when someone remembers.

T4 Formatter not enforced

Laravel Pint flags about 40 PHP files.

T5 Large parts of the app have no tests

Backend: order create/payment/cancel, reports, CSV import, users, settings, locations and warehouses, dashboard. Frontend: New Sale, Orders, Bill, Stock in/out, Reports, Users, Settings. No stock test ever uses binned stock, which is how C1 passed.

Working well

Verified, and worth keeping as the fixes land.

Race-safe stock Every quantity change is row-locked, re-read under the lock, and backed by a database <code>CHECK (quantity >= 0)</code>; the movement ledger is append-only.	Server-side money Prices are resolved from the catalogue at sale time; client-sent totals and "PAID" claims are ignored.
One permission gate All 74 routes pass one <code>permission:</code> middleware. Role confinement is verified by tests and live probes.	Session security httpOnly cookies with CSRF, sign-in that doesn't reveal which emails exist, and passwords and tokens redacted from the audit log.
Delete guards Categories, locations and warehouses holding parts or stock refuse deletion with a clear 409.	Hardened photo import Nothing is extracted by filename, contents must be real images, and files get random names — no zip-slip.
Double dispatch impossible 8 concurrent processes dispatching one order: exactly one succeeded, with one record and one audit entry.	Clean build ESLint clean, and 153 automated tests passing across backend and frontend.

Method. Automated suites, dependency audits, static review of every service and controller, ~40 crafted API requests, a real 8-process concurrency test, and a Chrome walkthrough. Destructive probes ran on the isolated test database on port 8001. The dev database was not modified.

Scope note. The audit includes the Security / Yard / Dispatch module built earlier in this session. Of its findings, only L5 and M9 ("dispatched today") apply to it.